Introduction to

Information Retrieval

Modified from Stanford CS276 slides

Chap. 2: The term vocabulary and postings lists

Exercise

1. **Why are skip pointers not useful for queries of the form x OR Y?**
2. **We have a two-word query. For one term the postings list consists of the following 16 entries:**

   [4,6,10,12,14,16,18,20,22,32,47,81,120,122,157,180]

   and for the other it is the one entry postings list:

   [47]

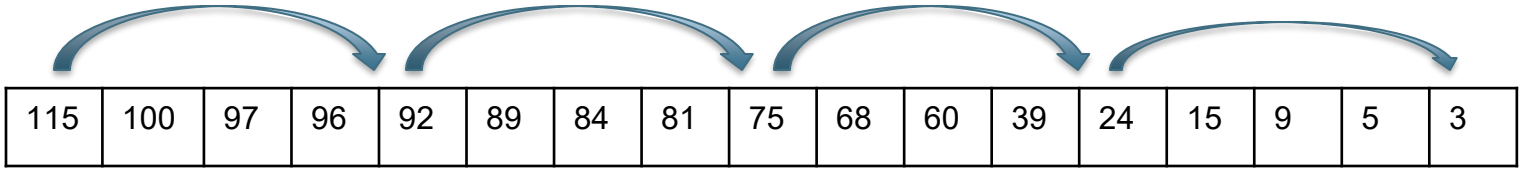
Work out how many comparisons would be done to intersect the two postings lists with the following two strategies. Briefly justify your answers:

A. **Using the standard posting lists**

B. **Using postings lists stored with skip pointers, with the suggested skip length of** $\sqrt{P}$

Exercise

3. **Consider a postings intersection between this postings list, with skip pointers**



And the following intermediate result postings list(which hence has no skip pointers):

101	100	99	97	95	89	5	3

Trace through the postings intersection algorithm

A. **How often is a skip pointer followed (i.e., p_1 is advanced to skip (p_1))**

B. **How many postings comparisons will be made by this algorithm while intersecting the two lists?**

C. **How many postings comparisons would be made if the postings lists are intersected without the use of skip pointers?**

Exercise

4. Shown below is a portion of the positional index in the format

term: doc1: position1,position2 …; doc2: position1, position2 …; etc.

angels:	2:36,174,252,651;	4:12,22,102,432;	7:17;
fools:	2:1,17,74,222;	4:8,78,108,458;	7:3,13,23,193;
fear:	2:87,704,722,901;	4:13,43,113,433;	7:18,328,528;
in:	2:3,37,76,444,851;	4:10,20,110,470,500;	7:5,15,25,195;
rush:	2:2,66,194,321,702;	4:9,69,149,429,569;	7:4,14,404;
to:	2:47,86,234,999;	4:14,24,774,944;	7:199,319,599,709;
tread:	2:57,94,333;	4:15,35,155;	7:20,320;
where:	2:67,124,393,1001;	4:11,41,101,421,431;	7:16,36,736;

Which document(s) if any meet each of the following queries, where each expression within quotes is a phrase query?

A. **“fools rush in”**

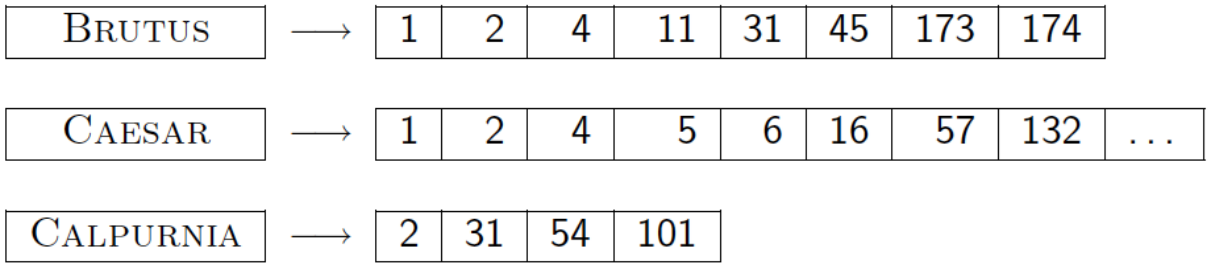
B. **“fools rush in” AND “angels fear to tread”**

Recap of the previous lecture

- The major steps in inverted index construction:
  - Collect the documents to be indexed
  - Tokenize the text
  - Do linguistic preprocessing of tokens
  - Index the documents that each term occur in

Recap of the previous lecture

- Basic inverted indexes:
  - Structure: Dictionary and Postings



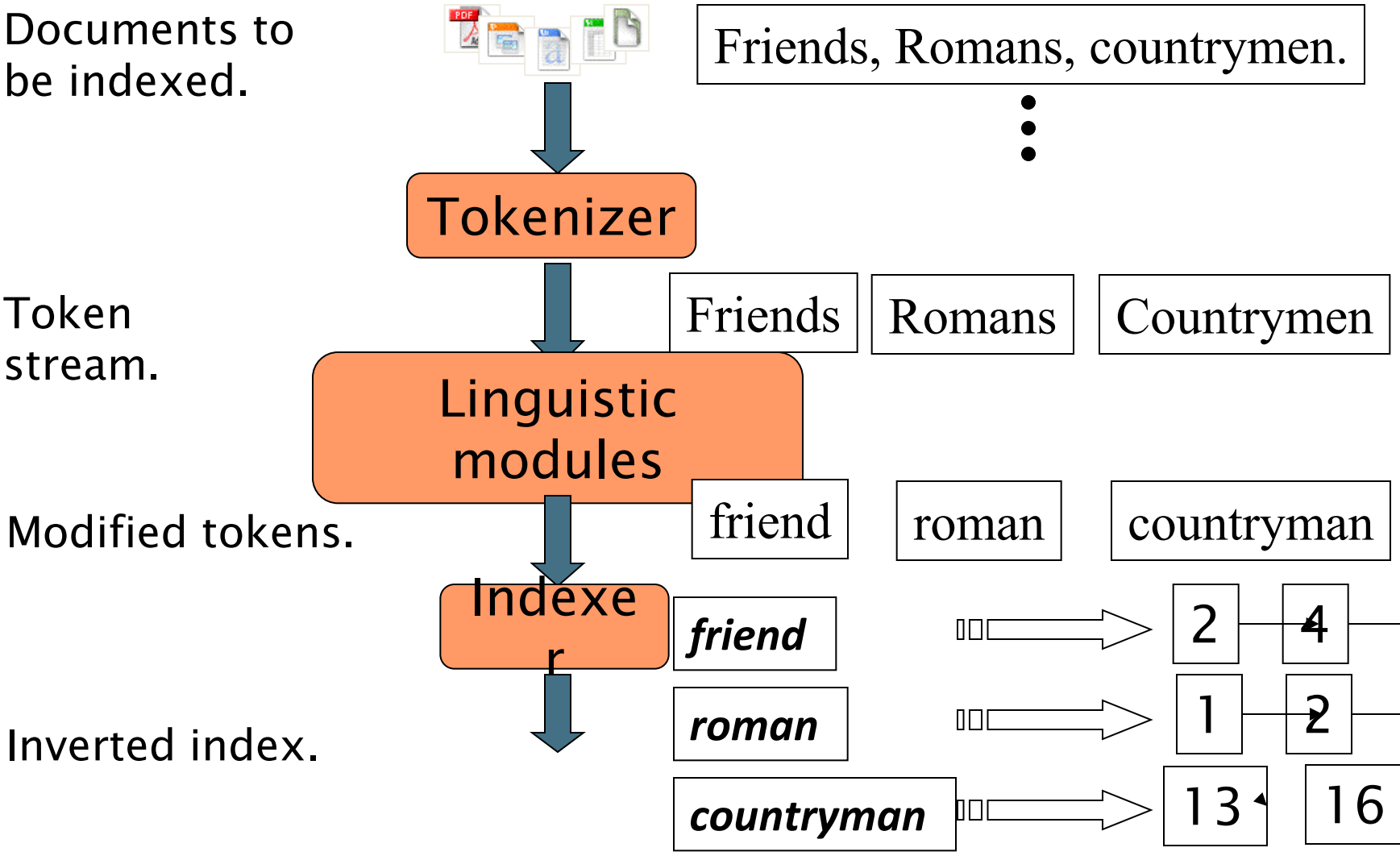
  - Key step in construction: Sorting
- Boolean query processing
  - Intersection by linear time “merging”
  - Simple optimizations

Plan for this lecture

Elaborate basic indexing

- Preprocessing to form the term vocabulary
  - Documents
  - Tokenization
  - What *terms* do we put in the index?

Recall the basic indexing pipeline



1. OBTAINING THE CHARACTER SEQUENCE IN A DOCUMENT

Parsing a document

- What format is it in?
  - pdf/word/excel/html?
- What language is it in?
- What character set is in use?

Each of these is a classification problem, which we will study later in the course.

But these tasks are often done heuristically …

Complications: Format/language

- Documents being indexed can be written in many different languages
  - A single index may have to contain terms of several languages.
- Sometimes a document or its components can contain multiple languages/formats
  - French email with a German pdf attachment.
- What is a unit document?
  - A file?
  - An email? (Perhaps one of many in an mbox.)
  - An email with 5 attachments?
  - A group of files (PPT or LaTeX as HTML pages)

TOKENS AND TERMS

Tokenization

- Is the task of chopping a character sequence and document up into pieces called tokens
- Throwing away a certain characters such as punctuation
- Tokens are referred to as terms or words
- A token is an instance of a sequence of characters in a particular document
- Each such token is now a candidate for an index entry, after further processing

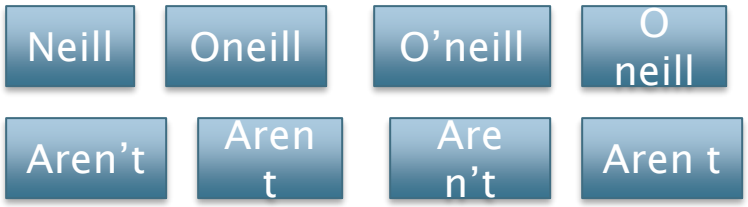
Tokenization

- Example:
- Input: "***Friends, Romans and Countrymen***"
- Output: Tokens
  - ***Friends***
  - ***Romans***
  - ***Countrymen***
- But what are valid tokens to emit?

Tokenization

- The major question of the tokenization phase is what are the correct tokens to use
- You chop on whitespace and throw away punctuation characters.
- **For example in English :**
  - ***What do you do about the various uses of the apostrophe for possession and contradictions?***

    Mr. O'Neill thinks that the boy's stories about Chile's capital aren't amusing
  - ***For O'Neill, which of the following is the desired tokenization?***



Tokenization

- Issues in tokenization:
  - ***Finland's capital →***

    Finland? Finlands? Finland's?
  - ***Hewlett-Packard*** → ***Hewlett*** and ***Packard*** as two tokens?
    - ***state-of-the-art***: break up hyphenated sequence.
    - ***co-education:*** delete
    - ***lowercase***, ***lower-case***, ***lower case*** ?
    - It can be effective to get the user to put in possible hyphens
  - ***San Francisco***: one token or two?
    - How do you decide it is one token?

Numbers

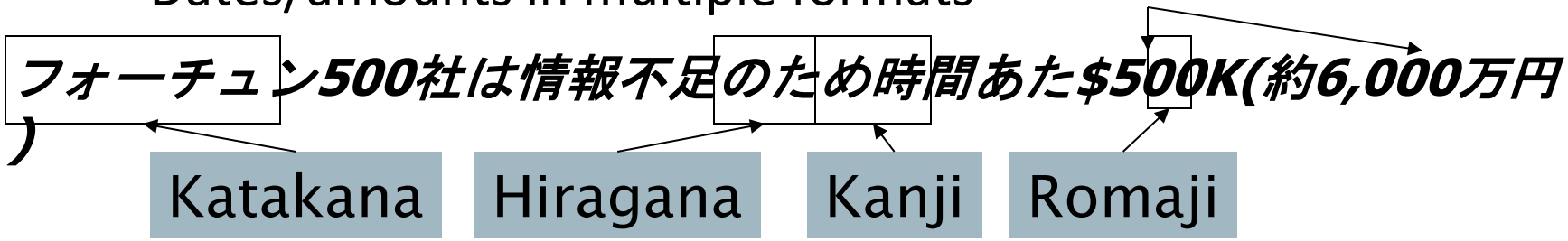
- ***3/20/91*** ***Mar. 20, 1991***
  20/3/91
- ***55 B.C.***
- ***B-52***
- ***My PGP key is 324a3df234cb23e***
- ***(800) 234-2333***
  - Often have embedded spaces
  - Older IR systems may not index numbers
    - But often very useful: think about things like looking up error codes/stacktraces on the web
    - (One answer is using n-grams: Chap. 3)
  - Will often index "meta-data" separately
    - Creation date, format, etc.

Tokenization: language issues

- French
  - ***L'ensemble*** → one token or two?
    - ***L*** ? ***L'*** ? ***Le*** ?
    - Want ***l'ensemble*** to match with ***un ensemble***
      - Until at least 2003, it didn't on Google
        - Internationalization!
- German noun compounds are not segmented
  - ***Lebensversicherungsgesellschaftsangestellter***
  - 'life insurance company employee'
  - German retrieval systems benefit greatly from a **compound splitter** module
    - Can give a 15% performance boost for German

Tokenization: language issues

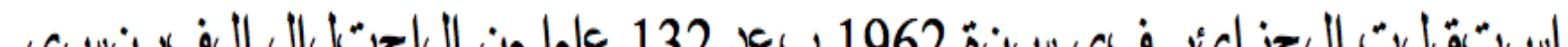
- Chinese and Japanese have no spaces between words:
  - 莎拉波娃现在居住在美国东南部的佛罗里达。
  - Not always guaranteed a unique tokenization
- Further complicated in Japanese, with multiple alphabets intermingled
  - Dates/amounts in multiple formats



End-user can express query entirely in hiragana!

Tokenization: language issues

- Arabic (or Hebrew) is basically written right to left, but with certain items like numbers written left to right
- Words are separated, but letter forms within a word form complex ligatures



- ← → ← → ←

start

- 'Algeria achieved its independence in 1962 after 132 years of French occupation.'
- With Unicode, the surface presentation is complex, but the stored form is straightforward

Stop words

- With a stop list, you exclude from the dictionary entirely the commonest words. Intuition:
  - They have little semantic content: *the, a, and, to, be*
  - There are a lot of them: ~30% of postings for top 30 words
- But the trend is away from doing this:
  - Good compression techniques (Ch. 5) means the space for including stopwords in a system is very small
  - Good query optimization techniques (Ch. 7) mean you pay little at query time for including stop words.
  - You need them for:
    - Phrase queries: “King of Denmark”
    - Various song titles, etc.: “Let it be”, “To be or not to be”
    - “Relational” queries: “flights to London”

Normalization to terms

- We need to "normalize" words in indexed text as well as query words into the same form
  - We want to match ***U.S.A.*** and ***USA***
- Result is terms: a term is a (normalized) word type, which is an entry in our IR system dictionary
- We most commonly implicitly define equivalence classes of terms by, e.g.,
  - deleting periods to form a term
    - ***U.S.A., USA*** ⍰ ***USA***
  - deleting hyphens to form a term
    - ***anti-discriminatory, antidiscriminatory*** ⍰ ***antidiscriminatory***

Normalization: other languages

- Accents: e.g., French ***résumé*** vs. ***resume*.**
- Umlauts: e.g., German: ***Tuebingen*** vs. ***Tübingen***
  - Should be equivalent
- Most important criterion:
  - How are your users like to write their queries for these words?
- Even in languages that standardly have accents, users often may not type them
  - Often best to normalize to a de-accented term
    - ***Tuebingen, Tübingen, Tubingen*** ⍰ ***Tubingen***

Normalization: other languages

- Normalization of things like date forms
  - ***7月30日 vs. 7/30***
  - ***Japanese use of kana vs. Chinese characters***

- Tokenization and normalization may depend on the language and so is intertwined with language detection

Morgen will ich in MIT …

Is this German "mit"?

- Crucial: Need to "normalize" indexed text as well as query terms into the same form

Case folding

- Reduce all letters to lower case
  - exception: upper case in mid-sentence?
    - e.g., ***General Motors***
    - ***Fed*** vs. ***fed***
    - ***SAIL*** vs. ***sail***
  - Often best to lower case everything, since users will use lowercase regardless of 'correct' capitalization…
- Google example:
  - Query ***C.A.T.***
  - #1 result is for "cat" (well, Lolcats) *not* Caterpillar Inc.



Normalization to terms

- An alternative to equivalence classing is to do asymmetric expansion
- An example of where this may be useful
  - Enter: ***window*** Search: ***window, windows***
  - Enter: ***windows*** Search: ***Windows, windows, window***
  - Enter: ***Windows*** Search: ***Windows***
- Potentially more powerful, but less efficient

Thesauri and soundex

- Do we handle synonyms and homonyms?
  - E.g., by hand-constructed equivalence classes
    - ***car*** = ***automobile*** ***color*** = ***colour***
  - We can rewrite to form equivalence-class terms
    - When the document contains ***automobile***, index it under ***car-automobile*** (and vice-versa)
  - Or we can expand a query
    - When the query contains ***automobile***, look under ***car*** as well
- What about spelling mistakes?
  - One approach is soundex, which forms equivalence classes of words based on phonetic heuristics
- More in lectures 3 and 9

Lemmatization

- Reduce inflectional/variant forms to base form
- E.g.,
  - *am, are, is → be*
  - *car, cars, car's, cars' → car*
- *the boy's cars are different colors → the boy car be different color*
- Lemmatization implies doing "proper" reduction to dictionary headword form

Stemming

- Reduce terms to their "roots" before indexing
- "Stemming" suggest crude affix chopping
  - language dependent
  - e.g., ***automate(s), automatic, automation*** all reduced to ***automat***.

for example compressed and compression are both accepted as equivalent to compress.



for exampl compress and compress ar both accept as equival to compress

Porter's algorithm

- Commonest algorithm for stemming English
  - Results suggest it's at least as good as other stemming options
- Conventions + 5 phases of reductions
  - phases applied sequentially
  - each phase consists of a set of commands
  - sample convention: *Of the rules in a compound command, select the one that applies to the longest suffix.*

Typical rules in Porter

- *sses* → *ss*
- *ies* → *i*
- *ational* → *ate*
- *tional* → *tion*

- Weight of word sensitive rules
- *(m>1) EMENT* →
  - *replacement* → *replac*
  - *cement* → *cement*

Other stemmers

- Other stemmers exist, e.g., Lovins stemmer
  - http://www.comp.lancs.ac.uk/computing/research/stemming/general/lovins.htm
  - Single-pass, longest suffix removal (about 250 rules)

- Full morphological analysis – at most modest benefits for retrieval
- Do stemming and other normalizations help?
  - English: very mixed results. Helps recall for some queries but harms precision on others
    - E.g., operative (dentistry) ⇒ oper
    - E.g., operational (research) ⇒ oper
    - E.g., operating (systems) ⇒ oper
  - Definitely useful for Spanish, German, Finnish, …
    - 30% performance gains for Finnish!

Language-specificity

- Many of the above features embody transformations that are
  - Language-specific and
  - Often, application-specific
- These are "plug-in" addenda to the indexing process
- Both open source and commercial plug-ins are available for handling these

Dictionary entries – first cut

ensemble.french
時間.japanese
MIT.english
mit.german
guaranteed.english
entries.english
sometimes.english
tokenization.english

